7 寸屏本机人机界面（显示屏 UI）需求规格

> 版本 v1.0 草案 · 2026-10-15 · 状态：**待用户确认，确认前不写任何 RTL** 配套：`docs/10_host_requirements.md`（上位机，已冻结 v1.0）、`docs/12_host_ui_requirements.md`（上位机 R1–R15 批次）、 `docs/RGB_LCD_wiring_check.md`、`docs/LCD_test_program_requirements.md`、`rtl/SPEC_interface_contract.md`、`docs/dual-channel-host-feasibility.md` 调研底稿：`7inch_UI_design_report.md`（UI/UX 调研）、`fpga_ui_feasibility_report.md`（FPGA 渲染可行性调研）

---

0. 一句话目标

不做第二个上位机，做一块"能脱离电脑独立演示"的本机人机界面：把编码器的倍频设置、转速/位置、 输入信号与 ABZ/UVW 波形、告警健康度，全部搬到 7 寸触摸屏上，用简洁明朗的中文界面呈现， 让评委在**不接笔记本**的情况下看到整机在工作。

1. 用户已确认的 6 条约束（2026-10-15，不得违背）

#	约束	来源
C1	**竞赛交付优先，屏是加分项，能做完多少算多少**	用户选择（工期问题）
C2	**屏 UI 和编码器做在同一片 FPGA 里**，用**内部寄存器总线**读写，不走 UART	用户选择（架构问题）
C3	**先花半天专门修触摸**；修好了 UI 就纯触屏（不设计按键兜底）	用户选择（触摸风险）
C4	**主界面中文（16×16 点阵）+ 数据用等宽数字**	用户选择（字体语言）
C5	**屏上不做数据记录**（记录仍由电脑上位机负责，屏只做"看"与"调"）	用户选择（数据记录）
C6	三条价值**都要**，不取舍：①脱离电脑独立演示 ②把标定/倍频/CORDIC 过程可视化给评委 ③工业现场 HMI 风格	用户自填

> C6 的含义：**页面集必须同时覆盖"演示抓眼球"（波形/李萨如/误差对比）、"数据可视化"（标定前后、角度误差曲线）和"工控面板"（告警/健康度/参数锁定）**。这不是加功能，而是三类页面各至少要有 一张：见 §7 的 P2/P3/P5（演示）、P7（可视化）、P6+P8（工控）。

1.1 工期事实（必须写进排期）

- 今天 **2026-10-15**；作品提交截止 **2026-11-04 18:00** ⇒ 剩 **20.8 天**。
- `docs/dual-channel-host-feasibility.md:321-348` 的 D1–D20 排期**已被必交物占满**： 大模型协作记录（15 分）、`skill/` 技能包（15 分）、文件名英文合规整改（52 处非 ASCII）、 设计报告定稿、干净目录复现验证、演示视频、英文海报。**这些一项都不能被屏挤掉。**

- ⇒ **屏的排期原则：只做 D10–D16 那个"可选加分项"窗口里能做完的部分，且必须能随时停在任意一页而不留半成品。** 每完成一页就单独出位流、上板验证、提交冻结，**绝不做"全部页面写完再一起上板"。**

2. 硬件基线（已实测/已核对，不是估计）

项	值	证据
器件	`xc7a35tfgg484-2`（Artix-7）20,800 LUT6 / 41,600 FF / 90 DSP48E1 / **50×RAMB36 = 1800 Kb = 225 KB / 5 CMT**	`docs/ATK_board_digest.md`
系统时钟	**50 MHz @ V4**，全工程单时钟域	`constr/enc_top.xdc:51-52`（唯一一条 `create_clock -period 20.000`）
屏	ATK-7016 7" 1024×600 IPS 电容屏（GT911），插 **J7**	`docs/RGB_LCD_wiring_check.md`
屏时序	PCLK **50 MHz**、H=20/140/1024/160（行总 **1344**）、V=3/20/600/12（帧总 **635**）、DE 模式、极性正	`lcd_test2/rtl/lcd_timing_gen.v:6-8`（指南表 13.1.3/13.4.2）
帧率	50e6 / (1344×635) = **58.60 Hz**	同上
像素节拍	**1 拍 = 1 像素**（`pixel_en` 门控，**不做分频时钟**）	`lcd_test2/rtl/lcd_timing_gen.v:10-21`
消隐窗口	每帧 239,040 拍 ≈ **4.78 ms 空闲**（可用作批处理渲染）	1344×635 − 1024×600
编码器现占用	LUT **5,737/20,800 = 27.6%**、FF 3,458 = 8.3%、**BRAM 0/50**、DSP 16/90、IOB 48/250	`_synth_work/util.rpt`
编码器时序	**WNS +2.475 ns**（sys_clk→sys_clk，9,424 端点）	`_synth_work/timing.rpt`
显示引擎实测成本	`lcd_test2` 显示侧 **2,769 LUT（13.31%）**、FF 1,064（2.56%）、BRAM 0、IOB 37	`_lcd_work/impl_lcd2/util.rpt`
~~⚠已有隐患~~ **已消除**	曾是 WNS = −2.181 ns / 18 个失败端点（全在 `u_pat/pixel_data_reg`）；根因是像素组合路径里的 24 位除法/取模，改成 BCD 帧计数 + 刻度查表后 **WNS = +2.870 ns / 0 失败端点**（净改善 5.07 ns）	`_lcd_work/impl_lcd2/timing.rpt`；教训见 `docs/LCD_test_program_readme.md` §9.3
触摸历史	GT911 仿真 PASS（锁定类型=1、点坐标正确）；实机 `0x814E` 空闲时 0x00、上电后读到过 `0x82`（ready + 2 点）。**2026-10-15 定位到两个真实缺陷**：① 点位解码按"每点 5 字节"（真片是 **8 字节/点**）导致点 1 起全错位；② 抬手后点数不清零（`nodata_cnt` 已修）。**仿真模型当时也按 5 字节摆数据，所以仿真 8/8 全绿却漏掉了这个 bug**	`docs/LCD_test_program_readme.md` §6/§9.6、`lcd_test2/tools/lcd2_touch_probe.py`
数据链路	遥测帧 18 B @10 kHz，2 Mbaud，**链路占用 90.7%，余量仅 9.3%**	`rtl/SPEC_interface_contract.md §1.13`

2.1 ★ 目标像素时钟定为 50 MHz（不用 MMCM）——已推翻调研报告的 51.2 MHz

调研报告推导的 51.2 MHz / 59 Hz 是**反推值，不是厂商值**。实测结论是 **50 MHz / 58.60 Hz**，且理由充分：

1. **50 MHz 就是板载晶振**，`sys_clk` 直接当 PCLK ⇒ **零 MMCM、零 IP、零跨时钟域。** 这一条保住了本工程"全系统单时钟域"的核心架构叙事（`docs/01_technical_solution.md:93`）， 而引入 51.2 MHz 会同时打破它，并给 20.8 天的工期加一次 CDC 验证。
2. **"2 的幂"不需要像素时钟是 2 的幂。**调研报告想要的"1 样本/像素"用**相位累加器**即可做到： 50 MSa/s ÷ 逐行 1024 像素 = **每像素恰好 48.83 个样点**，用 1024 累加步进即可无抖动重采样； 而所有 FFT/分箱窗口都可以**在 50 MHz 域内自由选点数**（FFT 点数是设计参数，与 PCLK 无关）。
3. **51.2 MHz 唯一能省下的是"重采样相位累加器"（约 34 bit）**，代价却是 1 个 CMT + 一次 CDC + 新的 XDC 时钟约束 + 重新收敛。**收益 34 bit 寄存器，代价一次时序/C DC 风险 —— 不划算。**

> 结论：**PCLK = sys_clk = 50 MHz，单时钟域，零 MMCM。** 上一版草案里"51.2 MHz 更能解决关键约束" 的判断**作废**（该结论建立在一个错误前提上：误以为"1 样本/像素"必须靠时钟频率相等）。

3. 显示引擎架构决策（三条硬结论，来自可行性调研 + 本地资源核算）

3.1 硬约束：全屏帧缓冲在本器件上物理不可能

1024×600 全屏缓冲：8bpp = 614,400 B = **133 个 RAMB36（器件的 2.67 倍）**；RGB565 = 267 个； 即使 1bpp 也要 614,400 bit = 16.7 个 tile。⇒ **不做全屏帧缓冲，这不是取舍而是硬约束。**

3.2 采用混合（Hybrid）渲染架构

```
程序化背景/网格/边框/仪表盘        ← 纯组合逻辑谓词，0 BRAM
+ Copper 逐扫描线寄存器表          ← 635×32 bit = 0.55 tile，每行读 1 次
+ 8×16 位图文本层（中文 16×16）    ← 字符单元表 + 字库 ROM
+ 逐列 min/max 包络（时域波形）    ← 1,024 列 × 24 bit = 0.67 tile/通道
+ 逐列 6-bit 状态（数字时序）      ← 1,024 列 × 6 bit = 0.17 tile
+ 仅 X-Y 李萨如图用 1bpp 位平面    ← 512×512 = 7.1 tile（X 非单调，必须真位图）
+ 单像素/拍固定延迟流水线合成      ← ~2,150 LUT / ~1,000 FF / 0–8 DSP
```

为什么时域波形不需要"上一个采样"（这是本方案最关键的实现结论）： 逐列存 ymin/ymax，渲染时与左邻列合并 `lo(x)=min(ymin[x],ymin[x-1])`、`hi(x)=max(ymax[x],ymax[x-1])`， 像素级判据只有 `lo(x) <= y <= hi(x)` —— **2 个比较器、零历史、1 拍 1 像素、完全流水。** 代价是缩得很远时丢失列内细节（用 min/max 而非"首/末采样"语义可保住毛刺）——**可接受。**

3.3 BRAM 预算（全部 50 个 RAMB36 中的 24–37 个，留 ≥13 个余量）

用途	估算
字符单元表（128×37 单元 × 8 bit）	1.03 tile
8×16 ASCII 字库（96 字）	1 tile
中文 16×16 字库（目标 300–500 字）	**3.5–6 tile**（32 B/字）

用途	估算
Copper 逐扫描线表	0.55 tile
时域包络 ×N 通道	0.67 tile/通道
数字时序状态（6 通道）	0.17 tile
李萨如 512×512×1bpp	7.1 tile
原始 ADC 采样环缓冲（可选，长捕获）	2–4 tile
读预取/双缓冲余量	3–5 tile
合计	**24（精简）～37（丰富）**

3.4 明确不做的技术路线（附理由）

不做	理由
DDR3 + MIG 做帧缓冲	MIG 引入 43 引脚 + 新时钟域 + 校准风险，而 UI 根本不需要它（BRAM 24–37/50 够用）。且它会占掉本机 5 个 CMT 里的 3–4 个。
QSPI Flash 存位图	只读 + 取引脚要动 `STARTUPE2`（有破坏配置的风险）；而全部素材只要 100–200 KB = 2–4 tile。
8bpp/16bpp 调色板帧缓冲	同样面积 1bpp 位平面省 8 倍；用"位平面 + 逐扫描线调色寄存器"（Amiga/SAGA 模型）。
逐像素 1 次 BRAM 读	单口 36.86 M/s vs 50 M/s ⇒ 端口占用 72%，是陷阱。正确做法：**32 bit 字预取，每 32 像素读 1 次 = 2.3% 端口占用。**
覆盖式模态盖住波形	反模式（§8 第 6 条）。编辑面板放**下半屏**，上半屏波形继续刷新。

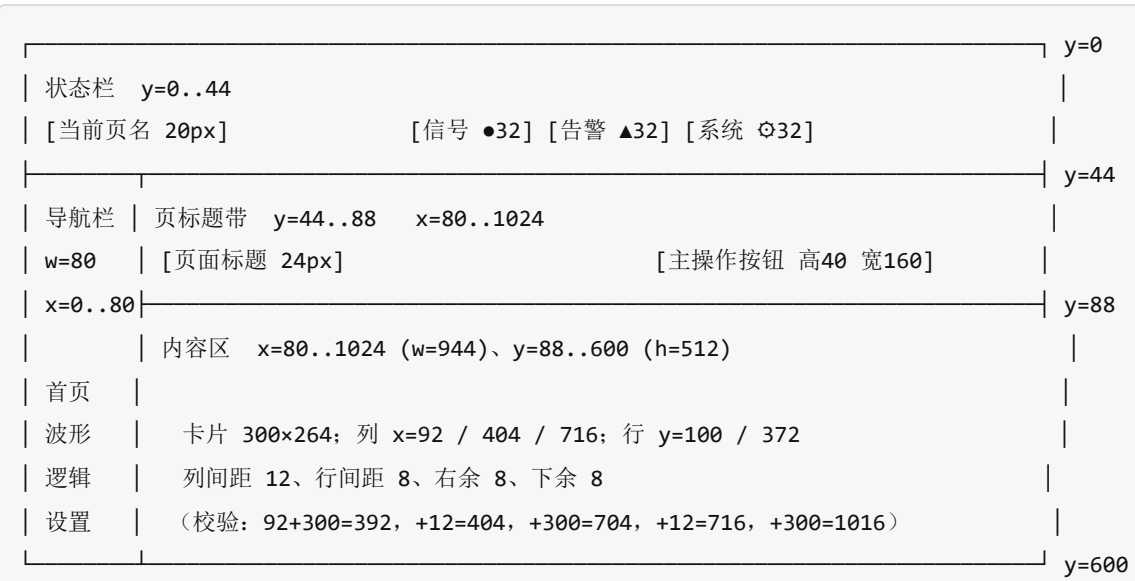
4. 信息架构（IA）

4.1 导航形态：左侧垂直导航栏（80 px）+ 页内 Tab

否决的方案	否决理由
底部标签栏	1024×600 是**宽屏**，纵向像素宝贵（600 px 里高度是稀缺维度）。底栏吃掉 56–72 px 高度，竖栏只吃 **7.8% 宽度。**
轮播/滑动翻页	导航入口被隐藏。仪表用户不接受"看不见入口"。
汉堡菜单/抽屉	同为隐藏入口；且多一次点击。
纯瓦片覆盖下钻	只留给**弹窗与编辑面板**，不做主导航。

顶层目的地 = 4 个 + 2 个溢出入口（状态栏 ⚙、卡片下钻），**信息架构深度 ≤ 2 层。**

4.2 公共骨架（1024×600，全部像素坐标）



- **导航项 = 80×88 px**（≥48 px 触控目标要求）。
- **内容区宽 944 px**（不是 1024−80−0，右侧留 8 px），**高 512 px。**

4.3 统一卡片模板（1 张 300×264 统计卡）



4.4 页面集（8 页；标 ★ 的是"C1 工期不足时可先砍"）

#	页面	入口	关键尺寸	对应 C6
P1	**总览**	导航「首页」	3×2 卡片（300×264）	演示 + 工控
P2	**输入波形**	导航「波形」→ Tab「模拟」	双通道 944×440（A y=140..340、B y=360..560）+ 右下 4 个徽标（Vpp/f/Δt/ΔV，各 88×32）	演示
P3	**李萨如 XY**	P2 → Tab「XY」	绘图区 560×560 居中（x=232..792，y=140..600）	演示
P4	**ABZ/UVW 时序**	导航「逻辑」	时序图 944×340（左标签栏 88、轨迹区 824）+ 时间轴	演示
P5	**六状态表 + 驻留统计**	P4 表格「展开」	6 行 × 5 列，行高 30、列宽 188，含占比条形图列	演示
P6	**设置 / 维护**	导航「设置」	3 列 × 4 项网格（项高 88）+ 底部危险区 944×96	工控
★P7	**标定与误差可视化**	总览卡「标定」下钻	左：标定前后误差曲线对比（944×300）；右：六参数当前值	**可视化（C6②）**

#	页面	入口	关键尺寸	对应 C6
★P8	**系统信息 / 自检**	状态栏 ⚙	设备 ID、固件版本、链路统计、资源占用、自检结果	工控

4.5 P1 总览页 6 张卡的字段分配

卡	主值（48 px）	次级（16 px）	可点?
1 输入频率	`1500.00` + `Hz`	倍频 ×64 分频 ÷1	—
2 **倍频 / 分频**	`×64` `÷1`	等效倍率 64 AB 边沿率 4×64×f_in	**整卡可点** → 弹出编辑面板
3 转速	`1800.0` + `rpm`	电气频率 1600.00 Hz 方向 CCW	—
4 角度 / 相位	`+45.0` + `°`	`pos_full 0x003A1B2C`	—
5 累计转数	`12 345 678` + `rev`	Z 脉冲计数	—
6 ABZ / UVW	`A● B● Z○` / `U V W`	底部迷你 6 通道逻辑条（每通道高 16）	—

> **卡片 2 整卡可点（命中区 300×264）→ 弹编辑面板**（不是新页面）：`[−] 64 [+]` + 离散滑条 + 数字键盘 + `取消` / `应用`。这是"总览页也能改参数"的关键，用户不必进设置页。

4.6 下钻路径（全部 ≤ 2 层）

```
P1 总览 ──点卡片2──→ 编辑面板（模态，只占下半屏，上半屏波形继续刷新）
        ──点卡片6──→ P4 逻辑时序 ──点表格标题"展开"──→ P5 六状态明细
        ──点卡片3──→ P7 标定与误差可视化
        ─导航「波形」→ P2 ─Tab「XY」→ P3
        ─导航「设置」→ P6 ──危险区:「清除锁存故障」「恢复默认」──→ 确认对话框
        ─状态栏 ⚙ ─→ P8 系统信息/自检
```

5. 排版与配色（沿用上位机已冻结色板，浅色为默认）

调研报告的深色方案**不采用**：`docs/10_host_requirements.md` 已冻结浅色色板，屏上位机保持视觉一致 才能真正实现 C6①"整机演示"。且明亮车间/展台强光下浅色更清楚、更省背光。

语义	颜色	用途
页面底	`#EAF3FB`	全局背景
卡片	`#FFFFFF`	卡片/面板
卡片描边	`#D6E3F0`	1 px 分隔
主强调	`#2E7DD1`	选中态、按钮、导航高亮
次级	`#8FC4EC`	次要图形
正文	`#1F2D3D`	主值、标题
次级文字	`#5B6B7F`	标签、单位、说明
正常	`#3FA45B`	● 正常

语义	颜色	用途
警示	`#E08B2D`	▲ 警示
故障	`#D9453D`	✕ 故障

字号（1-bit 字形光栅化，等宽 `DejaVu Sans Mono Bold`）

用途	字号
超大值（转速大数字，可选）	72–96 px
卡片主值	**48 px**
表格值 / 次主值	28–32 px
正文 / 标签	20 px
单位 / 次级	16 px
极小（下限）	14 px

5.1 三条排版铁律

1. **等宽（tabular）数字 + 固定小数位** —— 否则数值刷新时整行左右抖动（仪器 UI 最致命的业余感）。
2. **单位用 16 px 次级色，与数值基线对齐**，不缩放混排。
3. **每个页面最多 4 个字号、≤5 种颜色** —— 手写渲染器的成本控制，也是视觉秩序。

5.2 不用颜色也能区分状态（四重冗余，1-bit 渲染尤其重要）

`● 正常` / `▲ 警示` / `✕ 故障` = **字形 + 中文文字 + 反转视频（白字反色块）+ 可关闭的 2 Hz 闪烁。 颜色绝不作为唯一的状态编码**（色弱评委 + 强光下的真实问题）。

6. 交互规则（触摸仪器专用）

6.1 触控目标

类型	尺寸
最小（仅非关键项）	40 px
推荐（标准）	**48 px**
主操作	56–64 px
危险动作	72–96 px
相邻间距	≥8 px（理想 12 px）

6.2 ★ 两阶段提交（本需求最重要的一条交互契约）

任何写硬件寄存器的操作，都必须走：

```
改值 → [待应用 黄] → 点「应用」 → [应用中 …] → 回读寄存器确认 → [已应用 绿]
                                                  ↳ 不一致 → 自动回滚（红 + 错误码）
```

屏幕绝不允许显示"用户想要的理想值"，只能显示**寄存器回读确认过的实际值。** 理由：`rtl/SPEC_interface_contract.md §1.12` 已明确"读命令要连发两帧、写命令的应答内容是上一帧的没有语义"， 不设回读就无法区分"写成功"与"链路假死"。

6.3 倍频 `inter` (2…256) 用什么控件

连续滑条在 1024 px 上做 256 档 = 每档 <4 px，物理上不可用。 必须用： **预设档位步进器（2,4,8,16,32,64,128,256）+ 长按加速 + 离散滑条（档位吸附）+ 数字键盘兜底。** `div` (1…8) 用 8 档步进。编辑面板同时显示 `等效倍率 = inter/div` 与预测的 AB 边沿率，**改前就能看到后果。**

6.4 什么必须先确认（防误触）

必须二次确认	可以直接生效
清除锁存故障（`0x0405` 写 1 清）	页面导航
恢复出厂默认	Tab 切换
任何会改变 ABZ/UVW 输出的极性/方向位（`0x0109`）	查看类操作

6.5 长按 / 撤销

- **长按**只用于两件事：①步进加速 ②"防误触但不必弹窗"的动作（配环形成进度反馈）。 **长按绝不作为唯一入口**（用户不知道能长按 = 功能不存在）。
- **撤销**：编辑面板的「取消」= 不写寄存器（天然可撤销）。已写入的值不提供撤销，但**提供"上一次值"记忆**， 在面板上以"↩ 回到 ×8"的形式给出。

7. 数据来源与解算口径（不新增遥测帧，C2 直接读内部信号）

屏 UI 与编码器同片，**不需要串口。**所有数据直接从 `enc_top` 的内部信号与寄存器组读取。

屏上要显示的量	来源	口径与风险
倍频 / 分频	寄存器 `0x0101/0x0102`	`inter = {0x0102[1:0],0x0101[7:0]}` （0 视为 256）、`div = 0x0102[4:2]+1`。**改它们要同时写两个地址**
输入频率 `f_in`	①遥测帧的 `pos_full` ②**（推荐）RTL 新增频率计数器**	**★ 只拿 16 位 `angle` 求导在 >5 kHz 就混叠**（`host/encoder.py:10-12` 明确记录）。屏端**必须**用 `pos_full` 的 64 周期解回绕，或直接在 RTL 里数周期。
机械转速 RPM	`RPM = 60 × f_in / 极对数`（`0x0100`，默认 4）	标「估算值」。`POLE_BITS=6` 是虚拟 Z 口径，想按那套看就填 64 ⇒ `60×f_in/64`
方向	`pos_full` 增量符号，或 `0x0500` 的 dir 位	需要"中间断流后重新起算"保护（照抄 `host/encoder.py:124-128` 的做法）
累计转数	累计电周期数 ÷ 极对数	从"本次上电"起算，**不是掉电保存的绝对值**，屏上要写明
`s_n` / `c_n` 波形	遥测帧 off4..7 的 `s_n` / `c_n`（16 bit）	帧率 10 kHz ⇒ 时间分辨率 100 µs，是**软件极限**；屏上要标"等效采样"
原始 `sin/cos` 高速波形（可选②）	直连 `ad9238_capture` 的输出或 `sc_correct` 的输入	需新增采集环缓冲（2–4 tile）。**必须标"原始 50 MSa/s，非遥测口径"**，否则与上位机数字对不上

屏上要显示的量	来源	口径与风险
ABZ / UVW 六路电平	遥测帧 off9..11 的 `io_state` / `io_flags`	时间轴画 A/B 会**混叠**，必须按**电角度 64/128 箱**等效采样，并在图上注明
Z 重复率 / 脉宽、`dir` / `limiter` / `z_window`	遥测帧 `stat`、`io_flags`	—
幅值 / OTR	`0x0530` `STAT_AMP_SQ`、`0x053A` `STAT_FLAGS` bit3	`amp_sq` 是 (0.60)²…(1.20)² 门限口径（`0x0520/0x0524`）
告警与锁存	`0x0404` `STAT_VAL`（读一次同时得 `0x0405` 高字节）、`0x053A/0x053B`	★ **`0x0405` 是 W1C（写 1 清），与 iC-TW28"写 0 清位"相反**；写 bit0 还必须同拍发 `clr_otr`，否则 `led[0]` / `irq_n` 不恢复（2026-10-10 上板实测踩过）
标定状态与控制	`0x0502` `CALIB_CTRL`（start 写两次会触发两次标定）、`0x053A` bit4 caldone / bit5 calbusy	写 start 位**只发一帧**

7.1 UART 的处置（C2 推论）

- **屏的命令绝不走 UART**（余量只有 9.3%，一次寄存器读要 4 B 命令 + 6 B 应答，会把遥测挤爆）。
- **UART 遥测帧保持不变**（`AA 55` 头、18 B、10 kHz）⇒ **电脑上位机与屏可以同时工作**， 两端看到同一份数据，互不干扰。
- ⚠ **已确认的后果**：屏改了寄存器，**电脑上位机不会自动刷新**（遥测帧里不带配置回读字段）。 处理办法：屏改完可选**主动通过遥测帧的保留位广播一次"配置已变"**（`0x0101` 有保留位）； 若不做，就在文档里写明"两侧配置视图可能不同步，以回读为准"。→ **待确认项 Q6。**

7.2 渲染侧的时序约束（必须遵守，否则重蹈 −2.181 ns）

约束	数值/做法
像素流水线	**固定延迟、全寄存器化，1 拍 1 像素，无任何停顿**
单像素组合逻辑深度	≤4–8 级 LUT6（约 10–25 LUT 等效）
流水线寄存器宽度	40–56 bit/级 ⇒ 全 UI 约 300–450 FF
禁用	`char_code * 16 + y[3:0]` **绝不能组合出现在像素路径上**（必须预算/流水）
禁用	任意折线/多边形在像素路径上实时栅格化 ⇒ 一律**预栅格化进位平面**（消隐期或第二端口）
BCD/ASCII 转换	每帧在消隐期批处理一次（239,040 拍预算，double-dabble ~32 拍）
BRAM 访问	每平面 32 bit 字预取、**每 32 像素读 1 次**（2.3% 端口占用）
帧间撕裂	位平面**双缓冲，VBlank 交换**

8. 反模式清单（8 条，评审前逐条自查）

1. 把桌面 UI 按比例缩小到 1024×600。
2. **比例字体跑数字** ⇒ 刷新时整行抖动。
3. **颜色作为唯一状态编码。**

4. 装饰性渐变、圆角、阴影（手写渲染器的成本黑洞，且不增加信息量）。
5. 危险动作（清故障/恢复默认）混进常规列表。
6. **模态对话框盖住正在刷新的波形。**
7. 缺少"生效收敛态"——用户不知道值到底写进去没有。
8. 缺少空态/错误态（没信号时屏幕一片空白 / 一片 `0`）。

9. 里程碑（按 C1 可随时停在任意一页）

阶段	内容	产出判据
S0	**修触摸**（C3，见 §10）	实机上 `0x814E` 点数 > 0、坐标可读
S1	渲染骨架：`scanout_timing`(复用 `lcd_timing_gen`) + `region_decode` + `compositor` + `text_engine` + 中文点阵提取脚本	屏上出现中文标题 + 一个程序化方框，**WNS > +1 ns**
S2	**P1 总览页**（6 张卡，数据接 `enc_top` 内部信号）	卡片 1/3/4/5 显示真实数值；`f_in` 与上位机读数一致
S3	**P6 设置页 + 卡片 2 编辑面板**（含两阶段提交 + 回读确认）	屏上把 `inter` 从 64 改到 256，**回读确认**，AB 边沿率随之变化
S4	**P2 输入波形 + P3 李萨如**	≥10 次/秒刷新不卡；李萨如圆可见
S5	**P4 + P5 ABZ/UVW**	等效采样正确显示 90° 正交与六状态均衡
S6	★P7 标定与误差可视化 + ★P8 系统信息/自检	三相各自独立可砍
S7	打磨（配色/字号/长按/闪烁/空态）+ 集成到主位流 + 回归全绿	回归 21/21 PASS 且 `lint_all.ps1` ERROR 0

> **每阶段结束都必须：① `tools/lint_all.ps1` ERROR 0 ② 回归全量复跑 ③ 出位流并记录 WNS。** 任一阶段让 WNS 掉到负值或让回归变红 ⇒ **立即停下修，不进入下一阶段。**

10. 触摸修复方案（S0，半天）

已确认事实：`lcd_test2` 仿真曾 PASS（锁定类型 = 1（GT911）、点数 = 2、坐标 = (600,256)）， 生产代码里已有"按需读 + INT 快通道 + 10 ms 轮询"；实机空闲时 `0x814E` = 0x00。

> **2026-10-15 更新（重要，先读这段）**：多点问题已定位到**两个真实缺陷**，并已在 `lcd_test2` 里修掉：
> 1. **点位解码错位**：GT911 每个点占 **8 字节**（`0x814F + 8k`），旧代码按 **5 字节** 解码， 点 1 起全部错位（实机 `T2` 恒为 `(0,1025)`， `1025 = 0x401` 正是错位拼出来的假数）。 ⇒ 新渲染器**必须**按 8 字节/点解析，且**假从机模型也要按真片布局**， 否则仿真会继续"验证"错误假设（这次就是如此：仿真 8/8 全绿、真机多点全错）。 2. **抬手不清点**：点数据只在长读完成时锁存，`0x814E` 回到 0 后 `tch_num`/准星永远停在最后一帧。 ⇒ 已加 `nodata_cnt`（连续 ~0.5 s 无报点即清点数）。 上板复验用 `lcd_test2/tools/lcd2_touch_probe.py`（四阶段：静止/单指/抬手/多指）。 该工具能分清三种情形：芯片不报点 / 芯片报多点而 RTL 只认 1 点 / 芯片只报 1 点（配置表问题）。 细节见 `docs/LCD_test_program_readme.md` §6 与 §9.6。

> 注意：下面第 1 条（"先跳过配置区、只读 0x814E"）**当前实现已经是这样**（`CFG_WRITE_EN = 1'b0`），所以如果上板复验后 `0x814E` 在**手指按住时也一直是 0x00**，才需要走 F6 第 3 项（配置表写入）。
>
> **2026-10-15 第二次上板补充（比上面更要紧的一条）**：多点修复版烧进去后出现 "**手指一碰就永久停摆**"：`reads_ps` 掉 0、`tch_num`/坐标冻结、`err_ps` 也是 0。`err_ps=0` 说明 IIC 传输**从未结束**（不是 NACK/超时）。逐行穿读后确认 `i2c_master_seq.v` 的看门狗是**死代码**（判据把 25 位阈值截断成 16 位，且同一分支里 每拍把计数清零）⇒ "状态被从机拖住"就是不可恢复的锁死。已修： ① 看门狗改成**按状态停留时间**计数（`state_q` 比对，换状态清零），判据全宽且要求 `state == state_q`（否则刚换状态那一拍会用旧计数误触发——这个坑自检台抓到过）， 触发后进新增的 `S_RECOVER`（不看总线电平，纯打 10 个 SCL 时钟 + STOP，**保证**产生 `done`）； ② `touch_i2c_ctrl.v` 的 `ST_WAIT` 加 50 ms 兜底，超时强制回 `ST_IDLE + STEP_POLL`； ③ 遥测 byte4/5/6 的空位改为上报两套状态机的 `state/step/busy`（**§5.1**），卡死时直接读数定位； ④ 第 9 个自检台 `lcd_test2/sim/tb_i2c_watchdog.v` 把"卡住 → 恢复 → 下次请求正常"钉死。 **对 C2 渲染器的要求**：触摸状态机的等待**一律要有超时**，且把"最后一拍状态"暴露到调试通道； 不要写"等 `done` 一直等"的代码。细节见 `docs/LCD_test_program_readme.md` §6 第 6/7 条与 §9.7。

⇒ 这不像硬件坏，**最可能是四个初始化/时序细节之一。**排查顺序（按成本从低到高）：

1. **GT911 复位时序与 INT 电平**：GT911 靠 `INT` 在上电/复位期间的电平选择 IIC 地址（0x5D 或 0x14）。`lcd_test2` 探测到 `probe_gt=130`（GT911 有应答）说明地址选对了，但**必须确认 `0x8040` 的 控制寄存器（复位/上报率）与 `0x8047` 之后的配置区是否真的要重写**——GT911 出厂配置通常够用， **重写配置区反而可能导致 `0x814E` 不上报。★** 第一件事：**把配置区写入整个跳过，只读 `0x814E`**。
2. **`0x814E` 的读法**：GT911 是"读到点数后**必须写 0 清**"，且清之后要等下一轮扫描。 `lcd_test2` 若在同一轮里读完就清，下一次读可能仍是 0。
3. **中断/轮询的抢占**：INT 快通道与 10 ms 轮询若同时读 IIC，可能互相打断。 ⇒ 加一个"IIC 忙"互锁，并在 IIC 读失败时**不要**认为"点数 0"（区分 NACK/超时/真 0）。
4. **上拉电阻**：J7 排线上 `touch_scl` =T4 / `touch_sda` =U5 是否已有上拉（若靠 FPGA 内部弱上拉，2 Mbaud 下会失败； 但 IIC 只跑 100–400 kHz，通常够用）。

S0 的验收：实机上连续 100 次读 `0x814E`，出现点数 > 0 的次数 ≥ 90；触摸坐标与实际落点误差 ≤ 20 px。 **若半天内修不好**：按 C1 **当场降级** —— 用板载 `key_n`(W9) 做唯一输入，UI 退化为 "4 页循环 + 每页一个可调项"，**不做逐像素命中测试**（这条降级路线要预留，但按 C3 先不实现）。

11. 待确认项（确认前不写 RTL）

#	问题	我的建议
Q1	像素时钟用 **50 MHz**（零 MMCM，单时钟域，58.60 Hz）还是 51.2 MHz（+1 CMF/CDC，59.99 Hz）？	**50 MHz。**理由见 §2.1；51.2 MHz 的收益仅省一个相位累加器。
Q2	8 页全做，还是按 C1 工期先冻结 **P1+P6（总览+设置）** 这两页保交付，P2–P5/P7/P8 作为增量？	**先冻结 P1+P6。**这两页已经覆盖"屏上改倍频 + 看转速"这两个原始诉求。
Q3	中文字库做多少字？	**450 字**（覆盖全部页面文案 + 告警 + 常见汉字）≈ 14.4 KB = **3.5 tile**，余量充足。用 `tools/` 里新增的 Python 脚本从系统 TTF 光栅化生成 `font_cjk.v`。
Q4	屏幕上的"转速"由**RTL 新加频率计数器**算，还是屏幕端照抄上位机的 `pos_full` 解回绕算法？	**RTL 新加**（约 200 LUT + 1 个 32 bit 门控计数器）。理由：屏端算法要维护 64 周期解回绕状态机，且时序精度反而不如 RTL 直接数周期。

#	问题	我的建议
Q5	P7"标定与误差可视化"的数据从哪来？**现有遥测帧里没有"标定前后对比"数据**，需要 RTL 新增采集。	若要做 P7，需要新增一个"标定误差记录"寄存器区（`0x0540+`，16 个 16 bit 点 ×2 组前后对比）⇒ 约 +1 天 RTL + 0.5 天 UI。**建议 S6 再定，先不做。**
Q6	屏改了配置，要不要主动通知电脑上位机（用遥测帧保留位广播）？	**建议不做**，只在两边文档里写明"配置视图可能不同步，以回读为准"。做它要改 `uart_scope` 帧语义 ⇒ 属于 v13 版本变更。
Q7	★ 我建议先做一件调研报告没覆盖的事：**给 `lcd_test2` 的 −2.181 ns 失败端点做一次复盘**（18 个端点全在 `u_pat/pixel_data_reg`），把教训写进本文件的 §7.2，**再动手写新渲染器。**	**强烈建议做。**否则新渲染器很可能重蹈同一个坑，而这次是和编码器拼在一起，没收敛就交不出位流。

12. 本需求明确不做的事

- 不做数据记录 / SD 卡 / 导出（C5）。
- 不做 FFT 频谱（会挤掉 P2–P5 的工期，且演示价值低于时域波形 + 李萨如；**若工期富余再议**）。
- 不做多语言 / 双语切换（C4 定中文）。
- 不做屏幕亮度自动调节、屏保、动画过渡（装饰性，反模式 §8-4）。
- 不改动 `rtl/enc_top.v` 的既有端口与行为（只**新增**同片模块与内部信号引出）。
- 不改动 UART 遥测帧格式（除 Q6 另有决定）。
- 不碰 `host/`（上位机保持现状，屏与电脑可同时工作）。